Azure DevOps — Complete Study Notes

AZ-400 Aligned | Cognizant DevOps Training | All 6 Phases

PHASE 1 — The Mental Model

1.1 Why DevOps Exists — The Project Lifecycle Problem

To understand DevOps you have to go back to how software projects used to work. Every IT organization takes requirements from business users, builds an application, and then deploys it. That process needs a model — a structured way of moving from "someone wants a feature" to "feature is live".

The Waterfall Model

The earliest model borrowed from manufacturing and construction. It breaks a project into strict sequential phases. Each phase must be completed before the next begins. The model is like building a house — you pour foundations before you lay bricks, and you lay bricks before you put on the roof. You cannot go back.

- **Phase 1** — Project planning
- **Phase 2** — Requirements gathering
- **Phase 3** — Analysis
- **Phase 4** — Design (screens, data stores, architecture)
- **Phase 5** — Coding
- **Phase 6** — Testing
- **Phase 7** — Deployment

Advantages: clear demarcation between phases, all requirements captured upfront, project manager can estimate cost and delivery dates accurately.

Disadvantages: The customer only sees a working product at the END of the entire lifecycle — which could be 3 months to 2 years. By then a competitor product may already be available. Scope creep forces painful backtracking. Bugs found in testing can require returning all the way to design.

⚠ NOTE: The biggest Waterfall flaw: business requirements change over time, but the model cannot accommodate this without expensive rework of earlier phases.

The Agile Methodology

In 2001, 17 software developers met to address these problems and published the Agile Manifesto. The core idea: instead of one giant project, break it into small iterative cycles (Sprints). Each cycle goes through requirements → design → development → testing → deployment → review, and then repeats.

- **Iterative delivery** — Working software is delivered in increments, not at the end
- **Cross-functional teams** — Self-organizing teams from different functional areas (dev, QA, ops) work together
- **Constant collaboration** — Close coordination between teams, customers, and end users throughout — not just at requirements phase
- **Scrum** — A popular Agile framework: sprints (2-week cycles), sprint backlog, sprint reviews, daily standups

- **Kanban** — Another Agile framework: continuous flow, WIP limits, visualised board columns

How DevOps Was Born

Agile solved the development problem but a gap remained between the Development team (who builds features) and the Operations team (who deploys and maintains the infrastructure). Dev would hand off code to Ops, and Ops would struggle to deploy it. Blame culture existed between the two teams.

DevOps bridges this gap. It is a culture, a set of practices, and a toolchain that unifies development and operations — enabling faster, more reliable software delivery. The core practices are Continuous Integration (CI) and Continuous Deployment (CD).

- **CI (Continuous Integration)** — Developers merge code frequently; automated builds and tests run on every merge
- **CD (Continuous Deployment)** — Successfully built code is automatically deployed to test, staging, and production environments

1.2 The Azure DevOps Platform

Azure DevOps is a suite of tools from Microsoft available at dev.azure.com. It is NOT the same as the Azure portal (portal.azure.com) — it is a separate platform entirely. It covers the full DevOps lifecycle and aligns to the AZ-400 exam objectives.

The Five Core Services

Service	What it does	External Equivalent
Azure Boards	Plan and track work items, sprints, backlogs	Jira (Atlassian)
Azure Repos	Host Git repositories	GitHub
Azure Pipelines	Build and release pipelines (CI/CD)	Jenkins, Octopus Deploy
Azure Artifacts	Private package management	NuGet.org, npmjs.com
Azure Test Plans	Manual and exploratory testing	Third-party test tools

Organisation and Projects

The hierarchy in Azure DevOps is: Organisation → Projects → Repos/Boards/Pipelines/Artifacts. An organisation is typically your company or team. Within it you create one or more projects. Every tool (Boards, Repos, Pipelines, Artifacts) lives within a project.

- **Access levels** — Stakeholder (limited access), Basic (standard), Visual Studio Subscriber (premium)
- **Security groups** — Project Administrators, Contributors, Readers — assigned at project or organisation level
- **Permissions model** — Three tiers: Organisation level, Project level, Object level (individual repos, queries, dashboards)

⚠ NOTE: "Not set" permission is equivalent to "Denied". Users only get permissions explicitly granted to their security group.

Phase 1 — Summary Table

Concept	Key Point
Waterfall	Sequential phases, customer sees product only at end, hard to accommodate change
Agile	Iterative sprints, cross-functional teams, frequent delivery, constant collaboration
DevOps	Bridges Dev and Ops, CI/CD pipeline, automation, collaboration culture
Azure DevOps	Suite of 5 tools: Boards, Repos, Pipelines, Artifacts, Test Plans
Organisation	Top-level container; projects live inside it
Access Levels	Stakeholder / Basic / Visual Studio Subscriber
Permissions	Org level → Project level → Object level

PHASE 2 — Azure Boards + Source Control

2.1 Azure Boards

Azure Boards is the project management tool inside Azure DevOps. Think of it as Jira — you plan sprints, create work items, track progress, and report on team velocity. It supports both Agile and Scrum process templates.

Work Item Types

- **Epic** — Large body of work spanning multiple sprints — e.g. "Build the entire authentication module"
- **Feature** — A specific capability that delivers value — e.g. "Add password reset flow"
- **User Story** — A requirement from the user's perspective — e.g. "As a user I can reset my password via email"
- **Task** — The actual unit of work assigned to a developer — estimated in hours
- **Bug** — A defect in the application that needs to be fixed

The hierarchy is: Epic → Feature → User Story → Task/Bug. Each level breaks the work into smaller, manageable pieces.

Boards and Sprint Planning

The board shows work items as cards arranged in columns: New → Active → Closed. Teams can drag cards across columns as work progresses. Sprint planning involves pulling user stories from the backlog into a sprint, estimating effort, and assigning tasks. Capacity planning lets you set how many hours each team member has available per sprint.

Dashboards

Dashboards give a real-time project overview through configurable widgets. You can create multiple dashboards and share them across the team. Important built-in widgets include:

- **Burn Down Chart** — Tracks remaining work within a sprint — should reach zero by sprint end
- **Cycle Time** — Time taken to close a work item after work has started

- **Lead Time** — Time taken to close a work item from when it was first created
- **Velocity** — Team capacity to deliver work sprint over sprint
- **Sprint Burndown Widget** — Displays ideal vs actual burndown curve for the current sprint
- **Deployment Status Widget** — Shows pipeline deployment status — configured per pipeline and stage

Queries

Queries let you filter work items by fields like Work Item Type, State, Priority, Assigned To, and more. You can create complex multi-clause queries using AND/OR operators. Results can be charted (bar, pie, column charts) and added to dashboards.

- **My Queries** — Personal queries only visible to the logged-in user
- **Shared Queries** — Visible to all project members — requires Basic access level or higher to save

⚠ NOTE: To edit a Shared Query, you need to be a Project Administrator — OR have explicit Contribute permission granted on that specific query object.

Permissions in Azure Boards

Feature	Required Permission / Group
Create own queries (My Queries)	All users except Readers
Save to Shared Queries folder	Project Administrators OR explicit object-level Contribute permission
Edit a shared query	Project Administrators OR explicit object-level Contribute permission on that query
Create/edit dashboards	Project Administrators (or explicit dashboard permission granted)
View dashboards	All project Valid Users (default)

Wiki

Each Azure DevOps project includes a built-in Wiki. Pages are created using Markdown and are version-controlled in a dedicated Git repository behind the scenes. Teams use the Wiki for documentation, runbooks, onboarding guides, and architecture notes.

2.2 Git Fundamentals

Git is a distributed version control system. Every developer has a full copy of the repository including its history. Changes are tracked as commits. The most popular platform for hosting Git repositories is GitHub; within Azure DevOps it is Azure Repos.

Core Git Concepts

- **Repository** — A folder whose history is tracked by Git — can be local (on your machine) or remote (on GitHub/Azure Repos)
- **Commit** — A snapshot of changes — identified by a unique hash (SHA)
- **Branch** — A lightweight movable pointer to a commit — used to isolate work
- **Merge** — Integrating one branch into another
- **Pull Request (PR)** — A request to merge a feature branch into main — with code review

- **Tag** — A permanent label on a specific commit — used for releases (e.g. v1.0.0)
- **.gitignore** — A file listing paths/patterns Git should NOT track — e.g. build outputs, binary files

Essential Git Commands

```
git init                        # Create new local repo
git clone <url>                 # Download remote repo to local
git status                      # See which files changed
git add <file>                  # Stage file for commit
git add .                       # Stage ALL changes
git commit -m "message"         # Commit staged changes
git push origin main            # Push local commits to remote
git pull origin main            # Fetch + merge remote changes
git fetch origin                # Fetch without merging
git branch feature/login        # Create a new branch
git checkout feature/login      # Switch to that branch
git checkout -b feature/login   # Create + switch in one command
git merge feature/login         # Merge branch into current branch
git log --oneline               # View commit history
git stash                       # Temporarily save uncommitted work
git stash pop                   # Restore stashed work
git tag v1.0.0                  # Create a tag at current commit
git rebase main                 # Reapply commits on top of main
git cherry-pick <hash>          # Apply a single specific commit
git reset --hard HEAD~1         # Undo last commit (dangerous!)
git gc                          # Garbage collect unreferenced objects
git prune                       # Remove stale objects not referenced
```

Branching Strategies

A disciplined branching strategy is essential in team environments. The general principle is that main (or master) always contains production-ready code and feature work happens in separate short-lived branches.

- **main branch** — Always contains working production code — never commit broken code here
- **Feature branches** — Short-lived, one feature per branch, merged via pull request, then deleted
- **Release branches** — Used when delivering the same application to multiple customers with different requirements
- **Merge strategies** — Merge Commit (preserves history), Squash Merge (clean single commit), Rebase (linear history)

⚠ NOTE: Feature branches should be short-lived. Long-lived feature branches cause merge conflicts and integration nightmares. Merge early and often.

Large Files — Git LFS

Binary files (compiled builds, images, videos) should NOT be committed to Git. Git is designed for source code text files. For large files, Git LFS (Large File Storage) is used. It stores the large files externally and places pointer/reference files in the repository. Your repo stays lean while still being able to reference large assets.

⚠ NOTE: The build output (.dll, .exe) generated by Visual Studio should NOT be in version control — only the source code belongs in Git. That is why Visual Studio creates a .gitignore by default.

Git Maintenance

- **git gc** — Git automatically runs garbage collection to remove objects no longer referenced by any branch or commit
- **git prune** — Removes stale objects that are not reachable from any reference

2.3 Azure Repos

Azure Repos is the Git repository hosting service inside Azure DevOps. It works identically to GitHub in terms of pushing/pulling code, but it is integrated directly with Azure Pipelines, Azure Boards, and Azure Artifacts — making it the natural choice in an Azure-first environment.

- **Default repo** — Each project gets a default Git repo with the same name as the project
- **Multiple repos** — A single project can host multiple Git repositories
- **Branch Policies** — Rules enforced on protected branches — required before merging via PR

Branch Policies in Azure Repos

- **Minimum number of reviewers** — PR must be approved by N reviewers before merge is allowed
- **Check for linked work items** — PR must reference at least one Azure Boards work item
- **Check for comment resolution** — All PR review comments must be resolved before merge
- **Build validation** — A build pipeline must succeed against the PR code before merge is allowed

⚠ NOTE: Branch policies are enforced on the main branch to ensure code quality. They cannot be bypassed by regular contributors — only project admins can override.

Azure Repos vs GitHub

Feature	Azure Repos	GitHub
Hosting	Azure DevOps (dev.azure.com)	GitHub.com
Integration	Native Azure Pipelines, Boards, Artifacts	GitHub Actions, GitHub Projects
Branch policies	Yes (detailed)	Yes (branch protection rules)
Pull requests	Yes	Yes
Private repos	Yes (free)	Yes (free)
Best for	Azure-focused enterprise teams	Open source, GitHub-native workflows

Phase 2 — Summary Table

Concept	Key Point
Work Item hierarchy	Epic → Feature → User Story → Task/Bug
Burn Down Chart	Remaining work in sprint — target is zero by sprint end
Shared Queries	Visible to all — needs Basic access or Project Admin to save

git commit	Creates a snapshot (hash) of staged changes
git push	Sends local commits to the remote repository
Feature branch	Short-lived, one feature per branch, merged via PR
Git LFS	Stores large binary files externally with pointer in repo
Branch Policy	Rules on protected branches — reviewers, build validation, work item linking
.gitignore	Prevents build outputs and binaries from being tracked by Git

PHASE 3 — Build Pipelines (Continuous Integration)

3.1 What is a Build Pipeline

A build pipeline automates the process of taking source code from a repository, building it, running tests, and producing a deployable artifact. Think of it as a factory assembly line — code goes in one end, a tested, packaged build comes out the other.

Azure Pipelines supports two formats: YAML (code-based, stored in the repo) and Classic/GUI (drag-and-drop interface). YAML is the recommended modern approach because the pipeline definition lives alongside the code in version control.

3.2 YAML Pipeline Structure

Every YAML pipeline follows this structure. Understanding what each keyword does is essential for the AZ-400 exam.

```yaml
trigger:
  branches:
    include:
      - main

pool:
  vmImage: "ubuntu-latest"

variables:
  buildConfiguration: "Release"
  solutionPath: "**/*.sln"

stages:
  - stage: Build
    jobs:
      - job: BuildJob
        steps:
          - task: NuGetToolInstaller@1
          - task: NuGetCommand@2
            inputs:
              restoreSolution: "$(solutionPath)"
          - task: DotNetCoreCLI@2
            inputs:
              command: "build"
              projects: "**/*.csproj"
              arguments: "--configuration $(buildConfiguration)"
```

YAML Keyword Line-by-Line

`trigger:` → Defines what causes the pipeline to run — here it triggers on pushes to main

`pool: vmImage:` → Specifies which Microsoft-hosted agent to run jobs on (ubuntu-latest, windows-latest, macOS-latest)

`variables:` → Key-value pairs you can reference throughout the pipeline as $(variableName)

`stages:` → Top-level grouping — a pipeline can have multiple stages (Build, Test, Deploy)

`jobs:` → A stage contains one or more jobs — each job runs on its own agent

`steps:` → The actual list of tasks and scripts that make up a job

`task: NuGetCommand@2` → A built-in task — @2 is the version number. "inputs:" defines task-specific settings

3.3 Build Agents

Every pipeline job needs a machine to execute on. That machine is called an agent. Azure DevOps provides two kinds.

Microsoft-Hosted Agents

A fresh virtual machine is provisioned for each job. After the job completes, the VM is discarded. Microsoft handles all maintenance, security patches, and upgrades. This is the simplest approach — zero infrastructure management.

- **ubuntu-latest** — Ubuntu Linux — most common for .NET Core, Docker, Python
- **windows-latest** — Windows Server — required for .NET Framework, Visual Studio builds
- **macOS-latest** — macOS — required for iOS/Xcode builds

⚠ NOTE: The machine is destroyed after every job. Any builds, cached packages, or installed tools do NOT persist. Every run starts from a clean state.

Self-Hosted Agents

You create and manage your own VM (typically an Azure VM) and register it with Azure DevOps. The agent software runs on this VM and polls Azure DevOps for jobs. Use self-hosted agents when you need persistent build caches, custom software installations, or network access to on-premises resources.

Setup process for a self-hosted agent:

- **Step 1** — Create an Azure VM (e.g. Windows Server 2019 Datacenter)
- **Step 2** — Install required build tools on the VM (e.g. .NET SDK, Git, Java)
- **Step 3** — In Azure DevOps: go to Organisation Settings → Pipelines → Agent Pools → create or use Default pool
- **Step 4** — Generate a Personal Access Token (PAT) with Agent Pools Read/Manage scope
- **Step 5** — On the VM: download the agent zip from Azure DevOps, extract, run config.cmd, enter org URL and PAT
- **Step 6** — Run as a service — the agent is now registered and ready to accept jobs

⚠ NOTE: If your pipeline uses ubuntu-latest but your application targets .NET 6.0 and the hosted image does not have it, the build FAILS. Always verify the tools available on your chosen agent image.

3.4 Common Pipeline Tasks

Task Name	Purpose
NuGetToolInstaller@1	Installs NuGet tool on the agent

NuGetCommand@2	Runs NuGet restore/push/pack commands
DotNetCoreCLI@2	Builds, tests, or publishes .NET Core projects
VSBuild@1	Builds Visual Studio .NET Framework solutions
CopyFiles@2	Copies files to artifact staging directory
PublishPipelineArtifact@1	Publishes build output as a pipeline artifact for use in later stages
DownloadPipelineArtifact@2	Downloads an artifact published by an earlier stage/job
AzureCLI@2	Runs Azure CLI commands inside the pipeline
AzurePowerShell@5	Runs PowerShell against Azure resources
Bash@3	Runs a Bash script
PowerShell@2	Runs a PowerShell script
PublishTestResults@2	Publishes test results (JUnit, NUnit, MSTest) to the pipeline

3.5 Variables in Pipelines

Variables let you define reusable values within a pipeline. Instead of hard-coding a value in ten places, you define it once and reference it everywhere. Variables can be defined at the pipeline level, stage level, or job level — the most locally scoped definition wins.

```
# Define variables at pipeline level
variables:
  databaseServer: "myserver.database.windows.net"
  buildConfig: "Release"

# Reference a variable
steps:
  - script: echo $(databaseServer)

# Set variable at runtime from a script (makes it available to next steps)
- bash: echo "##vso[task.setvariable variable=myVar]newValue"

# Compile-time macro syntax (evaluated when pipeline is compiled)
- script: echo ${{ variables.databaseServer }}

# Runtime syntax (evaluated when script actually runs)
- script: echo $(databaseServer)
```

`$(variableName)` → Runtime syntax — value is substituted when the step executes

`${{ variables.name }}` → Compile-time (template) syntax — substituted before the pipeline runs

`task.setvariable` → Sets or changes a variable value at runtime — available to all subsequent steps in the same job

⚠ NOTE: Compile-time (${{ }}) uses the initial variable definition. Runtime ($()) picks up changes made by task.setvariable. This is a common exam question.

Variable Groups

Variable groups let you define variables once and reuse them across multiple pipelines. They are defined in Project Settings → Pipelines → Variable Groups. In your YAML you reference

them with the group: keyword. Crucially, you can link an Azure Key Vault to a variable group — the Key Vault secrets become pipeline variables automatically, with values masked in logs.

```
variables:
  - group: my-variable-group   # All variables in the group are now available

# After linking Key Vault to the group:
- script: echo $(myKVSecret)   # Value from Key Vault, masked in logs
```

3.6 Pipeline Artifacts

An artifact is the build output that travels from the Build stage to the Deploy stage. After building and publishing an application, you must store the output as a pipeline artifact so the release pipeline can pick it up and deploy it.

```
# Publish artifact from Build stage
- task: PublishPipelineArtifact@1
  inputs:
    targetPath: "$(Build.ArtifactStagingDirectory)"
    artifact: "myapp-artifact"

# Download artifact in Deploy stage
- task: DownloadPipelineArtifact@2
  inputs:
    artifact: "myapp-artifact"
    path: "$(Pipeline.Workspace)/myapp-artifact"
```

3.7 Code Quality — SonarQube

SonarQube is a static code analysis tool. It analyses your source code for bugs, code smells, security vulnerabilities, and duplication — without running the code. You integrate it into the build pipeline so every commit is analysed before the build succeeds.

- **Prepare Analysis** — Configures SonarQube on the agent before the build runs
- **Run Code Analysis** — Analyses the compiled code after the build step
- **Publish Quality Gate Result** — Reads the SonarQube quality gate result — pipeline fails if quality gate fails

Setup requires a SonarQube service connection in Azure DevOps pointing to your SonarQube server (could be self-hosted or SonarCloud).

3.8 Pipeline Reports and Flaky Tests

- **Pipeline Analytics** — Under your pipeline → Analytics tab — shows pass rate and duration per step across all recent runs
- **Flaky Tests** — A test that fails intermittently without code changes — often caused by external service dependencies
- **Flaky Test Detection** — Enabled under Project Settings → Test Management → Flaky test detection. Azure DevOps can automatically mark tests as flaky so they do not skew your pass percentage

⚠ NOTE: Flaky test detection is enabled for ALL pipelines by default in Azure DevOps. You can configure whether flaky tests are included or excluded from the test pass percentage.

Phase 3 — Summary Table

Concept	Key Point

YAML trigger	Defines what branch push starts the pipeline
pool vmImage	Specifies the Microsoft-hosted agent OS
Microsoft-hosted agent	Fresh VM per job, destroyed after — no persistence
Self-hosted agent	Your own VM, registered with PAT, persistent builds possible
task.setvariable	Changes a variable value at runtime within the pipeline
$() vs ${{}}	Runtime evaluation vs compile-time (template) evaluation
Variable Group	Shared variables across pipelines — can link Azure Key Vault
Pipeline Artifact	Build output passed from Build stage to Deploy stage
SonarQube	Static code analysis integrated into build pipeline
Flaky test	Test that fails intermittently without code change — auto-detectable

PHASE 4 — Release Pipelines + Infrastructure as Code

4.1 Release Pipelines

The release pipeline takes the artifact produced by the build pipeline and deploys it to target environments. A full CI/CD flow looks like: code push → build pipeline builds and tests → artifact is produced → release pipeline deploys to Test → then Staging → then Production.

Classic (UI) Release Pipeline

The classic editor is a canvas-based GUI where you define stages, tasks, and conditions visually. While YAML multi-stage is the modern approach, the classic editor is still widely used for release pipelines in practice.

- **Artifacts** — The source of what you are deploying — can be from a build pipeline, Azure Artifacts feed, or Git repository
- **Stages** — Logical environments — Deploy to Test, Deploy to Staging, Deploy to Production
- **Agent Job** — The unit of execution within a stage — runs on a hosted or self-hosted agent
- **Pre/Post-deployment conditions** — Gates and approvals that must be satisfied before/after a stage runs
- **Approvals** — A human must manually approve before the pipeline advances to the next stage
- **Gates** — Automated checks — e.g. Azure Monitor alerts must be zero, or a work item query must return no active bugs

Continuous Deployment Trigger

When enabled, the release pipeline triggers automatically every time a new build artifact is created. The lightning bolt icon on the artifact in the classic editor toggles this on. Without it, releases must be triggered manually.

YAML Multi-Stage Release Pipeline

In YAML, you can extend your build pipeline into a release pipeline by adding a second stage. The build stage produces an artifact and the deploy stage downloads and deploys it.

```yaml
stages:
  - stage: Build
    jobs:
      - job: BuildJob
        steps:
          - task: DotNetCoreCLI@2
            inputs:
              command: "publish"
          - task: PublishPipelineArtifact@1
            inputs:
              artifact: "webapp-artifact"

  - stage: Deploy
    dependsOn: Build
    jobs:
      - job: DeployJob
        steps:
          - task: DownloadPipelineArtifact@2
            inputs:
              artifact: "webapp-artifact"
          - task: AzureWebApp@1
            inputs:
              azureSubscription: "my-service-connection"
              appName: "my-webapp"
              package: "$(Pipeline.Workspace)/webapp-artifact/**/*.zip"
```

`dependsOn: Build` → Deploy stage only runs if the Build stage succeeded

`AzureWebApp@1` → Task that deploys a zip package to an Azure Web App

`azureSubscription:` → References the service connection name that authorises access to the Azure subscription

`$(Pipeline.Workspace)` → The root directory where all artifacts are downloaded on the agent

Service Connections

A service connection is a stored, secured set of credentials that allows Azure DevOps to talk to an external service — your Azure subscription, GitHub, Docker Hub, Kubernetes cluster, SonarQube server, etc. You define it once in Project Settings → Service Connections and reference it by name in every pipeline.

⚠ NOTE: NEVER embed credentials (passwords, connection strings) directly in a pipeline YAML file. Always use service connections. The credentials are stored securely and masked in logs.

4.2 Deployment Strategies

How you deploy your application to production matters. Different strategies trade off deployment speed against risk and downtime.

Strategy	How it works	Downtime?	Risk
Rolling	Gradually replace old instances with new —	Minimal	Medium — some users see old, some new during deploy

	one by one or in batches		
Blue-Green	Run two identical environments (Blue=live, Green=new). Switch traffic when green is verified	Zero	Low — instant rollback by switching back
Canary	Send a small percentage of traffic to new version first, then gradually increase	Zero	Low — limits blast radius of bad deploy

Blue-Green with Azure Load Balancer

In a blue-green setup you have two backend pools on an Azure Load Balancer. The Production pool (Blue) serves live traffic. The Staging pool (Green) receives the new deployment. Once verified, the load balancer rules are updated to point traffic to Green. Blue becomes the fallback for instant rollback.

4.3 Infrastructure as Code — ARM Templates

ARM (Azure Resource Manager) templates are JSON files that describe Azure infrastructure declaratively. Instead of clicking through the portal, you write a JSON file and Azure deploys exactly what you describe. This is the native Azure IaC approach.

ARM Template Structure

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-
01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "webAppName": { "type": "string" }
  },
  "variables": {
    "appServicePlanName": "[concat(parameters('webAppName'), '-plan')]"
  },
  "resources": [
    {
      "type": "Microsoft.Web/serverfarms",
      "apiVersion": "2022-03-01",
      "name": "[variables('appServicePlanName')]",
      "location": "[resourceGroup().location]",
      "sku": { "name": "F1", "capacity": 1 },
      "properties": { "name": "[variables('appServicePlanName')]" }
    },
    {
      "type": "Microsoft.Web/sites",
      "apiVersion": "2022-03-01",
      "name": "[parameters('webAppName')]",
      "location": "[resourceGroup().location]",
      "dependsOn": ["[resourceId('Microsoft.Web/serverfarms',
variables('appServicePlanName'))]"],
      "properties": {
        "serverFarmId": "[resourceId('Microsoft.Web/serverfarms',
variables('appServicePlanName'))]"
      }
    }
  ]
```

```
}
```

`"$schema"` → Identifies this as an ARM template — required first field
`"contentVersion"` → Version of your template — informational
`"parameters"` → Values passed in at deployment time — makes the template reusable
`"variables"` → Internal computed values — reusable within the template
`"resources"` → The actual Azure resources to create or update
`"type"` → Azure resource type — e.g. Microsoft.Web/sites for an Azure Web App
`"apiVersion"` → Version of the Azure API to use — must match documentation for that resource type
`"location"` → Azure region — using resourceGroup().location takes it from the resource group
`"dependsOn"` → Tells ARM to create this resource AFTER the listed resource — enforces deployment order
`"serverFarmId"` → Links the Web App to its App Service Plan via the plan resource ID

⚠ NOTE: The API version in your ARM template MUST match the version documented for that resource type. Different API versions may have different supported properties.

ARM Template Deployment Modes

- **Incremental (default)** — Only deploys/updates the resources defined in the template. Existing resources NOT in the template are left alone
- **Complete** — Deploys the template AND deletes any resources in the resource group that are NOT in the template

ARM Template in Azure Pipeline (YAML)

```yaml
- task: AzureResourceManagerTemplateDeployment@3
  inputs:
    connectedServiceName: "my-service-connection"
    resourceGroupName: "myResourceGroup"
    location: "North Europe"
    templateLocation: "Linked artifact"
    csmFile: "$(Pipeline.Workspace)/template-artifact/template.json"
    overrideParameters: "-webAppName $(webAppName)"
    deploymentMode: "Incremental"
```

4.4 Infrastructure as Code — Terraform in Pipelines

Terraform is HashiCorp's open-source IaC tool. It uses HCL (HashiCorp Configuration Language) rather than JSON. Terraform is cloud-agnostic (works with Azure, AWS, GCP) and uses a state file to track what has been deployed. When used in Azure Pipelines it needs authentication to your Azure subscription.

Authentication — Service Principal

Terraform authenticates to Azure via a Service Principal — an application identity in Azure Entra ID. The pipeline impersonates this service principal to create/modify resources.

- **Step 1** — Register an app in Azure Entra ID (App Registrations) — creates a service principal
- **Step 2** — Generate a client secret for the app
- **Step 3** — Assign the Contributor role to the service principal on your Azure subscription (IAM → Role Assignments)
- **Step 4** — Set ARM_CLIENT_ID, ARM_CLIENT_SECRET, ARM_TENANT_ID, ARM_SUBSCRIPTION_ID as pipeline variables

Terraform Pipeline Tasks

- **TerraformInstaller@0** — Installs the specified version of Terraform on the agent
- **TerraformTaskV4 — init** — Initialises Terraform — downloads providers, configures backend state storage

- **TerraformTaskV4 — plan** — Shows what changes Terraform will make — does NOT apply them
- **TerraformTaskV4 — apply** — Applies the planned changes — creates/modifies Azure resources

⚠ NOTE: Terraform state should be stored in Azure Blob Storage (backend = azurerm). This allows the state to be shared across pipeline runs and across team members.

4.5 Desired State Configuration (DSC)

DSC is a PowerShell feature for managing the configuration of machines. It ensures a machine STAYS in the desired state — if someone removes IIS from a server, DSC automatically reinstalls it. This prevents configuration drift.

- **Azure Automation** — The Azure service that hosts a DSC Pull Server — it manages the configuration and the nodes
- **Configuration script** — A PowerShell script that declares what the node should look like — e.g. IIS must be installed
- **Node** — A machine (Azure VM) registered with the Automation account to receive the configuration
- **Onboarding** — VMs are registered under Automation Account → State Configuration (DSC) → Nodes → Add

⚠ NOTE: DSC is about keeping infrastructure in a known state. It is the IaC equivalent for machine configuration, not resource provisioning. ARM/Terraform creates resources; DSC configures what runs on them.

Phase 4 — Summary Table

Concept	Key Point
Classic release pipeline	GUI canvas — stages, tasks, approvals, gates, artifacts
YAML multi-stage	Build + Deploy in one YAML file — dependsOn controls order
Service connection	Stored credentials for Azure DevOps to connect to external services
Blue-Green deployment	Two identical environments — zero downtime, instant rollback
ARM template	JSON file declaring Azure infrastructure — deployed via AzureResourceManagerTemplateDeployment task
Incremental vs Complete	Incremental: adds/updates resources. Complete: adds/updates AND deletes any resource not in template
Terraform in pipeline	Needs Service Principal auth — Init → Plan → Apply tasks
DSC	Prevents configuration drift — Azure Automation Pull Server manages node state

PHASE 5 — Azure Artifacts + Security

5.1 Azure Artifacts

Azure Artifacts is a private package management service. When multiple development teams within an organisation share reusable code libraries, instead of publishing them to public package registries (like NuGet.org or npmjs.com), they publish to a private Azure Artifacts feed. Other teams authenticate to the feed and consume the packages.

Supported Package Types

- **NuGet** — For .NET projects — packages from NuGet.org
- **npm** — For Node.js/JavaScript projects — packages from npmjs.com
- **Maven** — For Java projects — packages from Maven Central
- **Python** — For Python projects — packages from PyPI
- **Universal Packages** — Binary blobs — any file format, Azure-specific

Feeds

A feed is the container that holds your packages. Feeds have visibility and access controls.

- **Project-scoped feed (default)** — Only accessible by members of that specific Azure DevOps project
- **Organisation-scoped feed** — Accessible by all members across all projects in the organisation
- **Public feed** — Open to anyone on the internet — only available in public projects

⚠ NOTE: Public feeds can only be created within public Azure DevOps projects. Organisation-scoped feeds are for internal sharing across teams without per-project access restrictions.

Feed Views

- **@local (default)** — All packages pushed to the feed land here first
- **@prerelease** — Packages promoted to this view are identified as pre-release
- **@release** — Packages promoted here are considered stable/production-ready

Upstream Sources

Upstream sources allow your private feed to proxy public package registries. When a developer requests a package from nuget.org through the feed, the feed fetches and caches the package locally. Next time it is requested, it is served from the cache — not the internet.

This is useful in locked-down corporate environments where developers cannot access the internet directly, or where you want centralised control over which external packages are allowed.

- **NuGet.org upstream** — Automatically added when feed is created with "include public source packages" enabled
- **npmjs.com upstream** — Can be added for npm packages

Publishing a Package (Manual)

```
# 1. Create a NuGet config file (nuget.config) in your project root:
<configuration>
  <packageSources>
```

```
    <add key="my-feed"
value="https://pkgs.dev.azure.com/myorg/myproject/_packaging/my-
feed/nuget/v3/index.json" />
  </packageSources>
</configuration>

# 2. Pack the project into a .nupkg package
nuget.exe pack co-services.csproj

# 3. Authenticate (first time)
# Install credential provider from Azure Artifacts page

# 4. Push the package to the feed
nuget.exe push -Source "my-feed" -ApiKey az co-services.1.0.0.nupkg
```

Publishing a Package via Azure Pipeline

```
steps:
  - task: NuGetCommand@2
    inputs:
      command: "restore"

  - task: NuGetCommand@2
    displayName: "Pack package"
    inputs:
      command: "pack"
      versioningScheme: "byBuildNumber"  # Uses pipeline build ID as version
      packagesToPack: "**/*.csproj"

  - task: NuGetAuthenticate@1
    displayName: "Authenticate to feed"

  - task: NuGetCommand@2
    displayName: "Push to feed"
    inputs:
      command: "push"
      publishVstsFeed: "AgileProject/my-feed"
```

⚠ NOTE: When the pipeline fails with "lacks permission to push", go to the Feed Settings → Permissions and add the Project Collection Build Service and the [ProjectName] Build Service accounts with the Contributor role.

5.2 Security in Azure DevOps

Azure Key Vault Integration

Azure Key Vault stores secrets, keys, and certificates securely. In Azure DevOps you link a Key Vault to a Variable Group. The secrets in Key Vault then become available as pipeline variables. Their values are masked (replaced with ***) in all pipeline logs — they are never shown in plain text.

```
# Variable group links Key Vault — in pipeline YAML:
variables:
  - group: my-keyvault-group   # Contains secrets from Key Vault

steps:
  - script: |
      az sql db connect --server $(sqlServerPassword)   # secret from KV
    env:
      sqlServerPassword: $(sqlServerPassword)   # Passes secret as env variable
```

⚠ NOTE: Never print Key Vault secrets with echo in a script step. Even though they are masked in the logs, it is a security anti-pattern. Pass them as environment variables to the processes that need them.

Personal Access Tokens (PATs)

PATs are the inbound equivalent of service connections. Service connections let Azure DevOps talk OUT to Azure. PATs let external things (agents, scripts, tools) talk IN to Azure DevOps. A PAT is scoped — when you create it you choose exactly what permissions it has.

- **Self-hosted agent registration** — Agent uses a PAT with Agent Pools Read & Manage scope to register itself
- **Deployment group registration** — Same process — PAT with Deployment Group Read & Manage scope
- **CI/CD tools (Jenkins etc.)** — External CI tools triggering Azure DevOps pipelines can use a PAT

⚠ NOTE: PATs are like passwords with a scope and expiry. Treat them with the same secrecy as passwords. Copy the token immediately after creation — Azure DevOps will never show it again.

OWASP ZAP — Dynamic Security Testing

OWASP ZAP (Zed Attack Proxy) is a dynamic application security testing (DAST) tool. Unlike SonarQube which analyses static code, ZAP actually sends HTTP requests to your running web application and looks for common security vulnerabilities like XSS, SQL injection, and insecure headers.

In an Azure Pipeline, ZAP is run as an Azure Container Instance. The test report is written to an Azure File Share. The pipeline then downloads the report, converts it, and publishes it as test results.

- **Step 1** — Create Azure Storage Account and File Share to receive the ZAP reports
- **Step 2** — Create an Azure Container Instance running the owasp/zap2docker-stable image
- **Step 3** — Pass the target URL and mount the File Share — ZAP runs the baseline scan
- **Step 4** — Download the XML report from the File Share to the pipeline agent
- **Step 5** — Convert XML to NUnit format using an XSLT transformation
- **Step 6** — Publish test results using the PublishTestResults task

⚠ NOTE: ZAP reports "failures" for security issues found in the application. These appear as failed tests in the pipeline test results — giving security visibility directly in your pipeline dashboard.

SonarQube vs OWASP ZAP

Feature	SonarQube	OWASP ZAP
Type	Static Analysis (SAST)	Dynamic Analysis (DAST)
What it scans	Source code	Running web application
When in pipeline	During build (before deploy)	After deploy (against live app)
Finds	Code smells, bugs, security hotspots in code	HTTP-level vulnerabilities (XSS, injection, etc.)
Requires running app?	No	Yes

Phase 5 — Summary Table

Concept	Key Point
Azure Artifacts feed	Private package registry — project-scoped or org-scoped
Feed views	@local (default), @prerelease, @release
Upstream sources	Proxy and cache packages from public registries (NuGet.org, npmjs)
NuGetAuthenticate@1	Task that authenticates pipeline to the Azure Artifacts feed for push
Feed permissions for pipeline	Add Project Collection Build Service and [Project] Build Service as Contributor
Key Vault + Variable Group	Link Key Vault to variable group — secrets become masked pipeline variables
PAT	Inbound auth to Azure DevOps — scoped, create once, copy immediately
SonarQube (SAST)	Static analysis of source code — integrated in build stage
OWASP ZAP (DAST)	Dynamic scan of running app — reports go to File Share, published as test results

PHASE 6 — Monitoring, Instrumentation + Interview Favourites

6.1 Azure Monitor

Azure Monitor is the central monitoring service in Azure. It collects metrics and logs from every Azure resource and lets you visualise, query, and alert on that data. It is the go-to tool when something breaks in production.

- **Metrics** — Numerical time-series data — CPU percentage, HTTP request count, memory usage, etc.
- **Activity Log** — Record of all administrative operations on resources — who created, deleted, or changed what. Useful for debugging failed ARM deployments
- **Log Analytics workspace** — A data store that collects diagnostic logs from resources. You query it with KQL (Kusto Query Language). Application Insights also stores its data here by default
- **Custom log search** — Write a KQL query against logs in the workspace — used in alert conditions

Alert Rules

An alert rule watches a metric or log and fires when a condition is met. The setup has three components: Scope (which resource), Condition (what triggers), Action Group (what happens).

```
Alert Rule components:
  Scope       → Which resource to monitor (e.g. a VM, Web App, Storage Account)
```

```
  Condition   → Metric: CPU > 70% | Activity Log: resource deleted | Log: custom
KQL query
  Threshold   → Static: fixed value | Dynamic: AI-determined baseline
  Evaluation  → Frequency: how often to check (every 1 min, 5 min, etc.)
  Granularity → Aggregation window — take average over last 5 min or 15 min
  Action Group → What to do when alert fires
```

`Frequency of evaluation` → How often the alert rule runs its check — e.g. every 5 minutes

`Aggregation granularity` → The time window over which to aggregate data — e.g. average CPU over last 15 minutes

⚠ NOTE: Frequency is how often you CHECK. Granularity is how much data you look BACK at when checking. Frequency=5min, Granularity=15min means: every 5 minutes, evaluate the average of the last 15 minutes of data.

Action Groups

An Action Group defines what happens when an alert fires. Multiple alerts can share the same action group. Actions include: Email/SMS/Push notifications, Azure Function invocation, Logic App trigger, Automation Runbook, Webhook, ITSM Connector.

- **ITSM Connector** — Connects Azure Monitor to external IT Service Management tools like ServiceNow. When an Azure alert fires, it automatically creates an incident in ServiceNow. Useful for organisations that manage incidents externally to Azure.

6.2 Application Insights

Application Insights is the APM (Application Performance Monitoring) tool within Azure Monitor. It is specifically for monitoring your web applications — tracking requests, dependencies, exceptions, and user behaviour. When you create an Azure Web App, Application Insights can be enabled at the same time.

⚠ NOTE: Application Insights stores all its data in a Log Analytics workspace in the background. You can query Application Insights data directly from the Log Analytics workspace using tables like appRequests, appExceptions, appTraces.

- **Users analytics** — How many unique users visited, from which locations, on which devices
- **Sessions** — Groups of user interactions within a time window
- **User flows** — Visualises the paths users take through your application
- **Funnels** — Shows where users drop off in a multi-step process
- **Performance** — Shows all HTTP operations, average duration, and request count — helps identify slow endpoints
- **Smart Detection** — Automatically detects abnormal patterns in failure rates, response times, and dependency failures — no manual threshold needed

Smart Detection creates an Action Group called "Application Insights Smart Detection" automatically when you create the resource. You can add your own notifications to this group.

6.3 Container Probes (Azure Container Instances)

When deploying containers to Azure Container Instances using YAML (container groups), you can configure health probes to manage the container lifecycle.

- **Liveness Probe** — Checks whether the container is still alive and functioning. If the probe fails, the Container Instance RESTARTS the container automatically. Use this to recover from application crashes.

- **Readiness Probe** — Checks whether the container is READY to accept traffic. If it fails, the container is removed from the load balancer until it recovers. Use this when your app depends on a database being available before starting.

⚠ NOTE: Liveness = is the container alive? (restart if not). Readiness = is the container ready to serve? (remove from load balancer if not). These are direct equivalents to Kubernetes probes.

6.4 Azure DevOps Notifications

Azure DevOps has a built-in notification system. Under User Settings → Notifications you can see all default subscriptions (build complete, pull request created, etc.). You can create new custom subscriptions to trigger emails based on specific events. Notifications can be set at the user level, team level, or project level.

6.5 AZ-400 Interview Favourites

Frequently Asked Concepts

- **What is the difference between CI and CD?** — CI = Continuous Integration — automatically build and test every code commit. CD = Continuous Deployment/Delivery — automatically deploy tested code to environments.
- **What is an Azure Pipeline agent?** — A compute resource (VM) that executes pipeline jobs. Microsoft-hosted = fresh VM per job, managed by Microsoft. Self-hosted = your own VM, registered with a PAT.
- **What is a service connection?** — Stored credentials in Azure DevOps that allow pipelines to connect to external services (Azure subscription, GitHub, Docker registry, Kubernetes cluster).
- **What is a deployment group?** — A logical group of on-premises or cloud VMs to deploy to. Each VM runs an agent that registers via a PAT and polls Azure DevOps for deployment jobs.
- **What is the difference between ARM and Terraform?** — Both are IaC tools. ARM templates are Azure-native (JSON), supported natively by Azure. Terraform is cloud-agnostic (HCL), has a cleaner syntax, and uses a state file. Both can be used in Azure Pipelines.
- **How do you pass secrets securely in a pipeline?** — Use Azure Key Vault linked to a Variable Group. Secrets are pulled at pipeline runtime, masked in logs, and never stored in the YAML file.
- **What is SonarQube's role in a pipeline?** — Static code analysis. It runs Prepare → Build → Analyse → Publish Quality Gate. The pipeline fails if the quality gate fails.
- **What is OWASP ZAP?** — Dynamic Application Security Testing (DAST). Runs an HTTP scanner against the deployed application. Results published as test results in the pipeline.
- **What is Application Insights Smart Detection?** — AI-powered anomaly detection. Automatically alerts on abnormal failure rates, response times, and memory leaks without manual thresholds. Creates a dedicated Action Group automatically.
- **What is the difference between Liveness and Readiness probes?** — Liveness = container alive? (restart if failing). Readiness = container ready for traffic? (remove from load balancer if failing).
- **What is a flaky test?** — A test that fails intermittently without code changes — usually due to external service dependencies. Azure DevOps can automatically detect and mark them, preventing false pipeline failures.

- **What are feed views in Azure Artifacts?** — @local = all packages. @prerelease = pre-release packages. @release = stable production packages. Used to promote packages through release gates.

Common Comparison Tables for Interviews

Feature	Microsoft-Hosted Agent	Self-Hosted Agent
VM lifecycle	Fresh VM per job, destroyed after	Persistent VM, stays running
Maintenance	Managed by Microsoft	Managed by you
Custom software	Limited to pre-installed tools	Install anything you need
Build caching	None (fresh every time)	Cache persists between runs
Network access	Internet only	Can reach on-premises resources
Cost	Free tier then pay-per-minute	Azure VM cost + your time to maintain

Feature	YAML Pipeline	Classic (GUI) Pipeline
Definition stored	In the Git repository alongside code	In Azure DevOps database
Version control	Yes — full history in Git	Limited (audit log only)
Code review	Yes — PR for pipeline changes	No
Portability	High — YAML can be reused	Low — tied to the project
Recommended for	New pipelines — modern standard	Legacy or release pipelines
Template reuse	Yes — YAML templates	Limited

Feature	Azure Repos	GitHub
Native integration	Azure Boards, Azure Pipelines	GitHub Actions, GitHub Projects
Branch protection	Branch policies	Branch protection rules
Best for	Azure enterprise environments	Open source, GitHub-native
Pull request reviews	Yes	Yes
Free private repos	Yes	Yes

Phase 6 — Summary Table

Concept	Key Point
Azure Monitor	Central monitoring — metrics, activity log, log search, alert rules

Alert condition types	Metric | Activity Log | Custom Log Search | Resource Health
Frequency vs Granularity	Frequency = how often to check. Granularity = how much data to look back at
Action Group	What to do when alert fires — email, webhook, runbook, ITSM
Application Insights	APM for web apps — users, performance, smart detection, stored in Log Analytics
Smart Detection	Auto anomaly detection — creates its own Action Group on resource creation
ITSM Connector	Azure Monitor → ServiceNow/etc — auto-creates incidents from Azure alerts
Liveness Probe	Is container alive? Restart if failing
Readiness Probe	Is container ready for traffic? Remove from load balancer if failing
Notifications	User Settings → Notifications — custom subscriptions for pipeline/repo events

End of Azure DevOps Study Notes

These notes cover all topics aligned to the AZ-400 exam across all 6 phases. Good luck, Allen.